Búsquedas inteligentes

René Castro

Inteligencia Artificial
Docente: José Mario Rios Félix
6-7pm

Tecnológico Nacional de México
Instituto Tecnológico de Culiacán

8-Puzzle

8-Puzzle es un ejercicio de un tablero 3x3. Se uso los algoritmos búsqueda en anchura, búsqueda en profundidad, búsqueda de costo uniforme, búsqueda A* y búsqueda A* de profundidad iterativa.

Se ejecuto con el siguiente estado inicial:

1	5	4
2	3	6
8	7	

Y se definió la solución como un puzzle 3x3 en orden (1, 2, 3, 4, 5, 6, 7, 8, <blanco>). Los resultados de la ejecución son los siguientes:

Algoritmo	Tiempo	Nodos expandidos	Longitud de la solución
BFS	95.32ms	168528	23
DFS	11.59ms	32701	18491
UCS	181.34ms	237459	103
A*	79.1ms	151068	103
IDA*	9.10ms	42160	23

Con otro estado inicial:

	8	5
7	1	6
4	3	2

Algoritmo	Tiempo	Nodos expandidos	Longitud de la solución
BFS	121.03ms	207992	23
DFS	47.81ms	157920	91113
UCS	208.17ms	303860	109
A*	124.89ms	204780	109
IDA*	15.82ms	54482	23

Un 8-Puzzle se puede resolver con cualquier algoritmo, pero con las pruebas se demuestra que tanto como la búsqueda en profundidad como la búsqueda A* con profundidad iterativa son los más rápidos para resolverlo y expanden la menor cantidad de nodos, pero la búsqueda en profundidad tiene una solución muy larga.

24-Puzzle

24-Puzzle es el mismo ejercicio pero usando un tablero 5x5. En este caso, la cantidad de permutaciones que se generen es mucho mayor, por lo que usar algoritmos como el BFS, DFS, UCS o A* no son óptimos.

Para este, se sugiere usar la búsqueda A* de profundidad iterativa, ya que el beneficio de esta es un uso mucho menor de memoria, ya que no guarda los nodos que explorará próximamente (esto hace el A* inadecuado), pero aumenta el costo de procesamiento para encontrar la solución. Para ello, también se usarán funciones de heurísticas para mejorar el tiempo requerido para encontrar la solución.

Se usarán las heurísticas de distancia de Manhattan y la de conflicto lineal. Para el ejemplo, se usará el siguiente puzzle:

1	2	3	4	5
6	7	8	13	10
11	12	9	14	15
20	17	18	19	16
21	22	23	24	

Heurística usada en IDA*	Tiempo	Nodos expandidos	Longitud de la solución
Distancia Manhattan	8.84s	48166111	41
Conflicto Lineal	1.87s	3844493	41

Hacer uso de la heurística de conflicto lineal permite encontrar la solución de forma mucho más optimizada que el de distancia Manhattan.

En el puzzle, es muy difícil encontrar la solución cuando dos celdas, una al lado de otra, en el puzzle actual, están en el puzzle objetivo en sus lugares opuestos, por lo que la heurística de conflicto lineal analiza estos casos y le agrega una "penalización" al costo estimado a la solución: distancia de Manhattan + penalización determinada por la heurística de conflicto lineal).

El código y un video demostrando su ejecución y explicación de las heurísticas se encuentran en el repositorio de GitHub de la clase: https://github.com/rcastro39/SCC1012/tree/main/03-puzzle#readme